LAPORAN PRAKTIKUM
TEKNOLOGI IOT II

MODUL 7: INTEGRASI BACKEND SERVER LOKAL (DOCKER & MOSQUITTO) DAN RULE ENGINE (NODE-RED)

Disusun Oleh :

Nama : Sekar Maulia

NPM : 25782112

Kelas : TRI - 3D

Dosen Pengampu :

Dr.Ir. Septafiansyah Dwi Putra, S.T., M.T.

Agus Ambarwari, S.Pd., M.Kom.

TEKNOLOGI REKAYASA INTERNET

JURUSAN TEKNOLOGI INFORMASI

POLITEKNIK NEGERI LAMPUNG

2026/2027

7.5 Praktikum

7.5.1 Praktikum 1: Pembangunan Ekosistem IoT Server dengan Docker Compose

Langkah Kerja:

i. Buat direktori bernama D:/iot-server, lalu di dalam folder iot-server, buat struktur sub-direktori berikut

```
D:\>mkdir iot-server

D:\>cd iot-server

D:\iot-server>mkdir mosquitto\config
```

ii. Buka teks editor (VS Code atau Notepad), buat berkas baru bernama mosquitto.conf, simpan di dalam folder iot-server/mosquitto/config/, lalu masukkan konfigurasi parameter berikut:

```
# ================================================================================
# KONFIGURASI DASAR ECLIPSE MOSQUITTO BROKER
# ================================================================================

# Listener Saluran MQTT Standar (Untuk NodeMCU ESP8266 & Klien TCP)
listener 1883
protocol mqtt

# Listener Saluran WebSockets (Untuk Aplikasi Web Browser & Dashboard Visual)
listener 9001
protocol websockets

# Perizinan Akses Awal (Tanpa Password)
allow_anonymous true

# Konfigurasi Persistensi Penyimpanan Pesan
persistence true
persistence_location /mosquitto/data/

# Konfigurasi Berkas Log
log_dest file /mosquitto/log/mosquitto.log
log_dest stdout
```

iii. Kembali ke folder induk iot-server, buat berkas baru bernama docker-compose.yml, lalu salin konfigurasi layanan terpadu berikut:

```
services:
  # ============================================================
  # 1. Layanan Broker MQTT: Eclipse Mosquitto
  # ============================================================
  mqtt:
    image: eclipse-mosquitto:latest
    container_name: mqtt-broker
    restart: unless-stopped
    ports:
      - "1883:1883"  # Port MQTT murni untuk mikrokontroler
      - "9001:9001"  # Port WebSockets untuk dashboard web
    volumes:
      - ./mosquitto/config:/mosquitto/config
      - ./mosquitto/data:/mosquitto/data
      - ./mosquitto/log:/mosquitto/log

  # ============================================================
  # 2. Layanan Visual Rule Engine: Node-RED
  # ============================================================
  nodered:
    image: nodered/node-red:latest
    container_name: node-red
    restart: unless-stopped
    ports:
      - "1880:1880"  # Port antarmuka web builder Node-RED
    environment:
      - TZ=Asia/Jakarta
    volumes:
      - ./nodered:/data
    depends_on:
      - mqtt
```

iv. Eksekusi perintah berikut untuk mengunduh (*pull*) *image* dan menjalankan kontainer di latar belakang (*detached mode*):

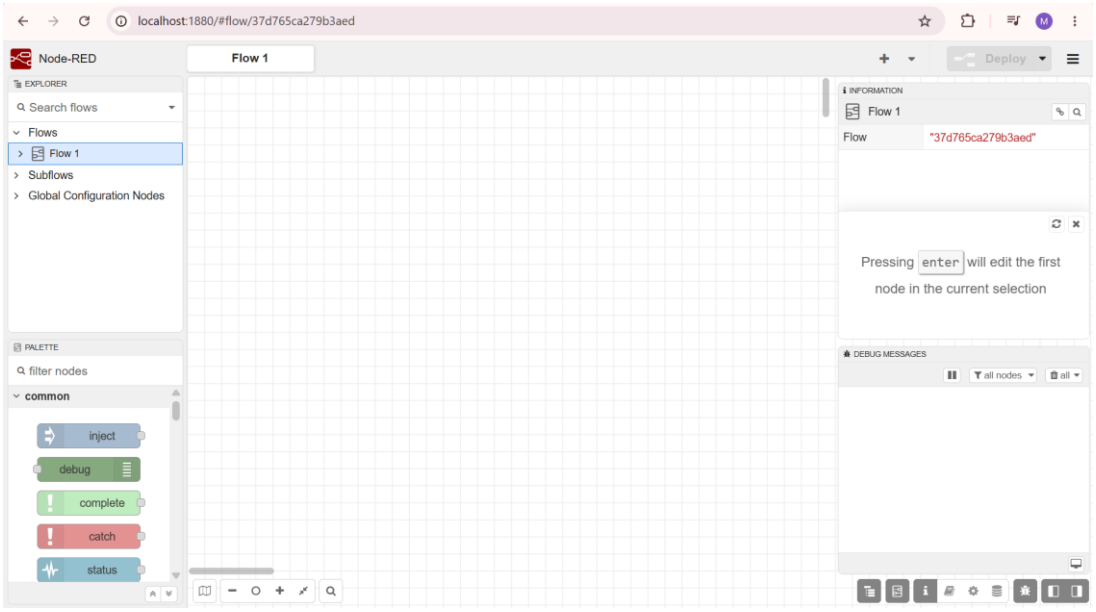
```
D:\iot-server>docker compose up -d
[+] up 29/29
 ✔Image eclipse-mosquitto:latest Pulled                    114.2s
 ✔Image nodered/node-red:latest  Pulled                    475.3s
 ✔Network iot-server_default     Created                     0.1s
 ✔Container mqtt-broker          Started                     3.1s
 ✔Container node-red             Started                     1.3s
```

v. Verifikasi bahwa kedua kontainer telah aktif dan berjalan normal dengan mengecek daftar proses kontainer:

```
D:\iot-server>docker ps
CONTAINER ID   IMAGE                    COMMAND                  CREATED        STATUS
          PORTS
   NAMES
fc248c4ff6ef   nodered/node-red:latest   "./entrypoint.sh"        19 hours ago   Up 54 seconds (h
ealthy)   0.0.0.0:1880->1880/tcp, [::]:1880->1880/tcp
   node-red
1449a84d8f4e   eclipse-mosquitto:latest   "/docker-entrypoint.…"   19 hours ago   Up 54 seconds
          0.0.0.0:1883->1883/tcp, [::]:1883->1883/tcp, 0.0.0.0:9001->9001/tcp, [::]:9001->9001/tcp
   mqtt-broker
065f74664ef5   wordpress:latest          "docker-entrypoint.s…"   7 days ago     Up 54 seconds
          0.0.0.0:8050->80/tcp, [::]:8050->80/tcp
   wordpress
3b95cb592631   mysql:latest              "docker-entrypoint.s…"   7 days ago     Up 54 seconds
          3306/tcp, 33060/tcp
   mysql-wp

D:\iot-server>
```

vi. Buka peramban web pada laptop Anda dan ketikkan alamat URL: http://localhost:1880.



7.5.2 Praktikum 2: Migrasi Simpul NodeMCU ESP8266 ke Broker MQTT Lokal

Perintah ipconfig, dan disini saya hanya menampilkan bagian wi-fi, karen akoneksi nya juga menggunakan wifi(wireless)

```
Wireless LAN adapter Wi-Fi:

   Connection-specific DNS Suffix  . :
   Link-local IPv6 Address . . . . . : fe80::1e02:c00a:6fc:aa87%20
   IPv4 Address. . . . . . . . . . . : 172.20.83.244
   Subnet Mask . . . . . . . . . . . : 255.255.255.0
   Default Gateway . . . . . . . . . : 172.20.83.177
```

Kode program

7.5.2_Praktikum_2.ino

```
/*
 * ========================================================================
 * PRAKTIKUM 7: SIMPUL SENSOR-AKTUATOR IOT UNTUK BROKER LOKAL
 * ========================================================================
 * Program ini membaca sensor DHT11/22, menyusun paket telemetri JSON, mengirimkannya
 * ke Broker Mosquitto Lokal di Laptop, serta mendengarkan instruksi kendali LED.
 * ========================================================================
 */

#include <ESP8266WiFi.h>
#include <PubSubClient.h>
#include <ArduinoJson.h>
#include <DHT.h>

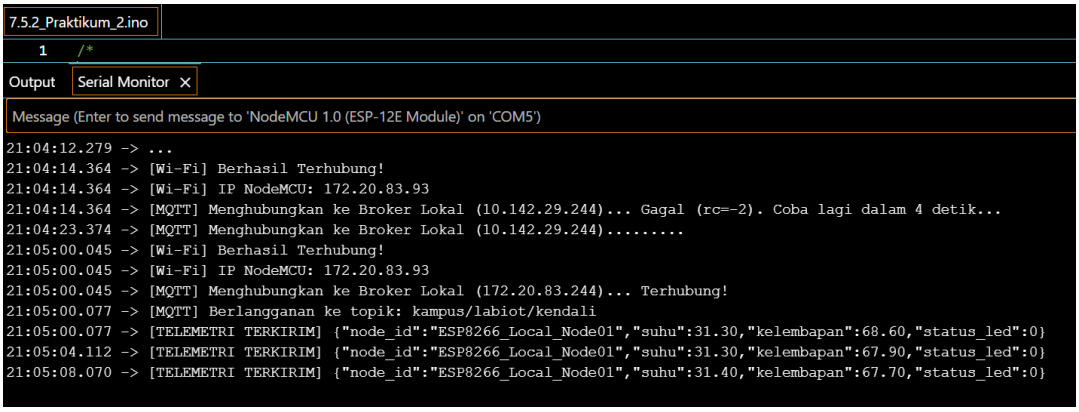
// 1. Kredensial Wi-Fi (Pastikan Laptop dan ESP8266 berada di Jaringan yang Sama!)
const char* ssid     = "kar";
const char* password = "12345678";

// 2. Konfigurasi Broker MQTT Lokal
// Ganti dengan Alamat IPv4 Laptop Anda yang didapat dari perintah 'ipconfig'!
const char* mqttServer = "172.20.83.244";
const int   mqttPort   = 1883;

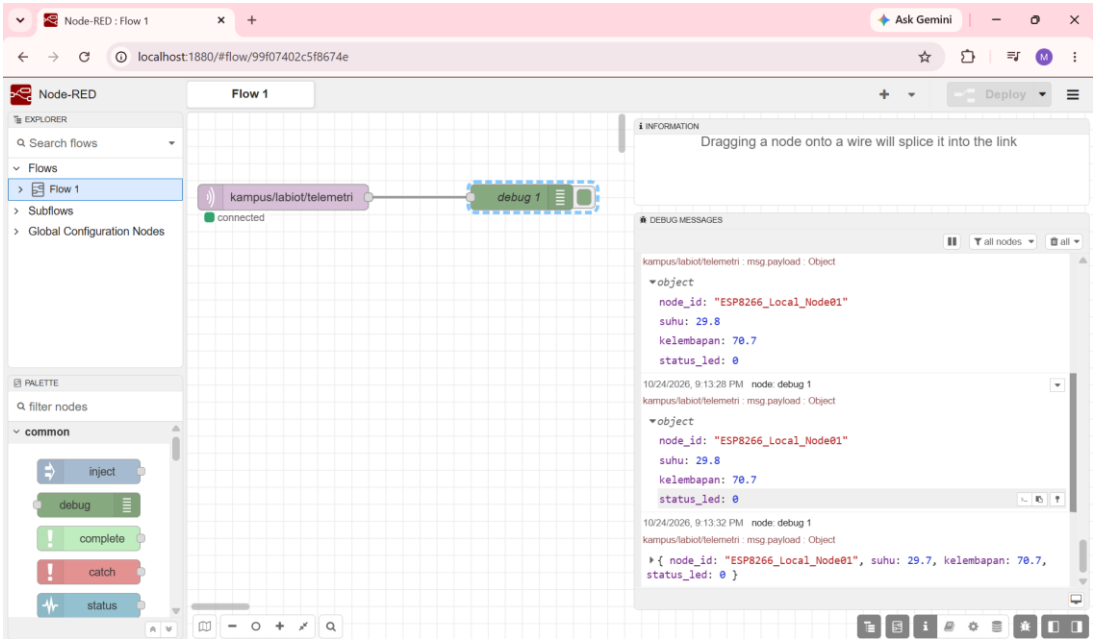
// 3. Konfigurasi Identitas Simpul & Topik
const char* clientId      = "ESP8266_Local_Node01";
const char* topicPubTele  = "kampus/labiot/telemetri";
const char* topicSubCmd   = "kampus/labiot/kendali";

// 4. Alokasi Pin Perangkat Keras
```

Serial monitor



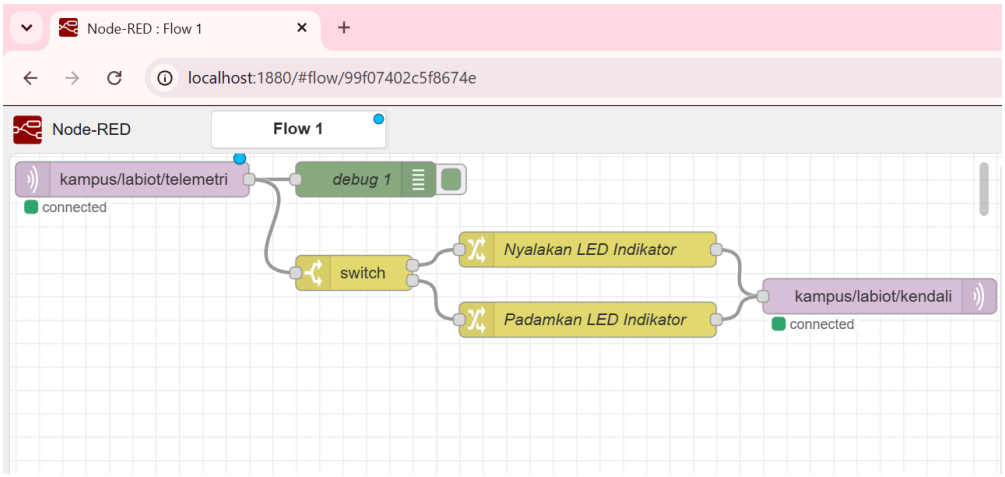
7.5.3 Praktikum 3: Integrasi Alur Data Telemetri Dua Arah pada Node-RED



Pada praktikum ini, Node-RED berhasil dihubungkan dengan broker Mosquitto lokal menggunakan node mqtt in. Topik kampus/labiot/telemetri berhasil menerima data dari NodeMCU ESP8266 dan menampilkannya melalui node debug. Data yang diterima berupa objek JSON yang berisi node_id, suhu, kelembapan, dan status_led. Status connected pada node MQTT menunjukkan bahwa komunikasi antara Node-RED dan broker Mosquitto telah berhasil berjalan dengan baik.

7.5.4 Praktikum 4: Implementasi Rule Engine Otomasi Aktuator Loop Tertutup

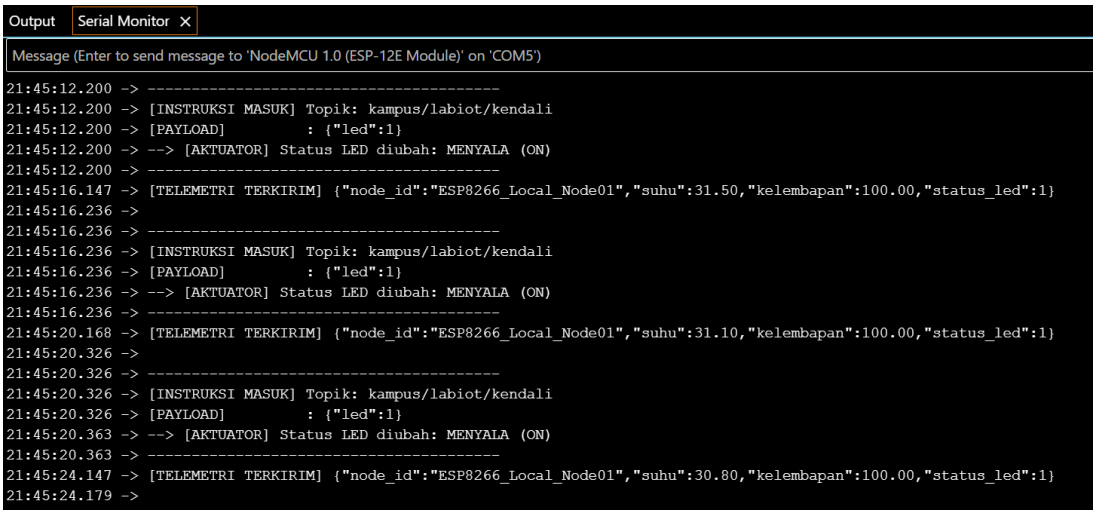
Kanvas Alur Lengkap: Closed-Loop Industrial Rule Engine (Web Node-RED)



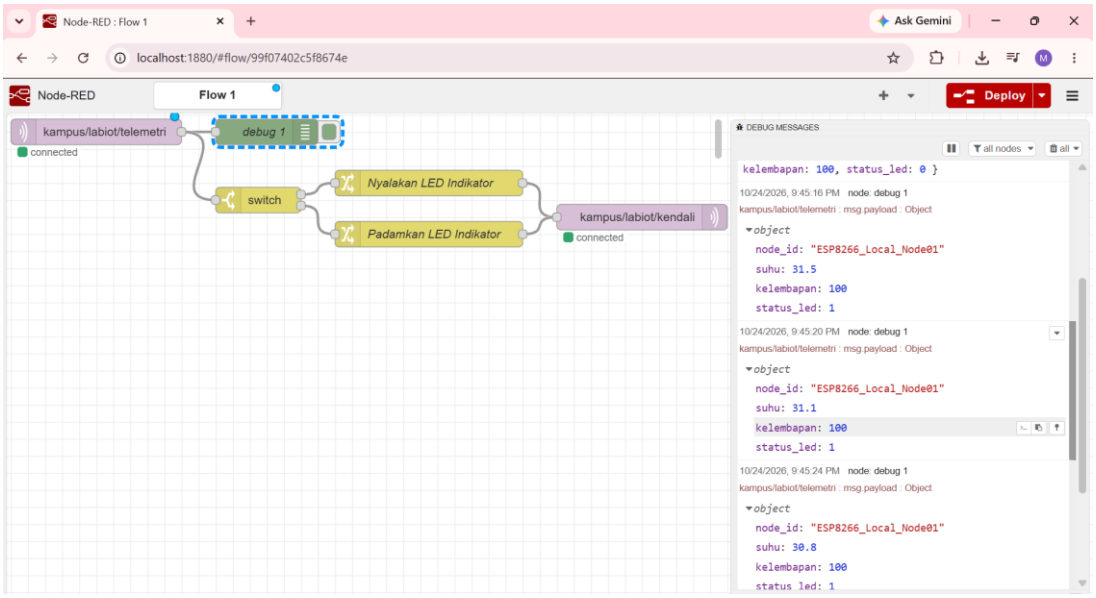
Hasil Pengamatan

- Seketika suhu melampaui 31.0°C, Node-RED mengeksekusi port 1, mengirimkan payload {"led": 1}, dan LED indikator bahaya di pin D1 seketika menyala terang

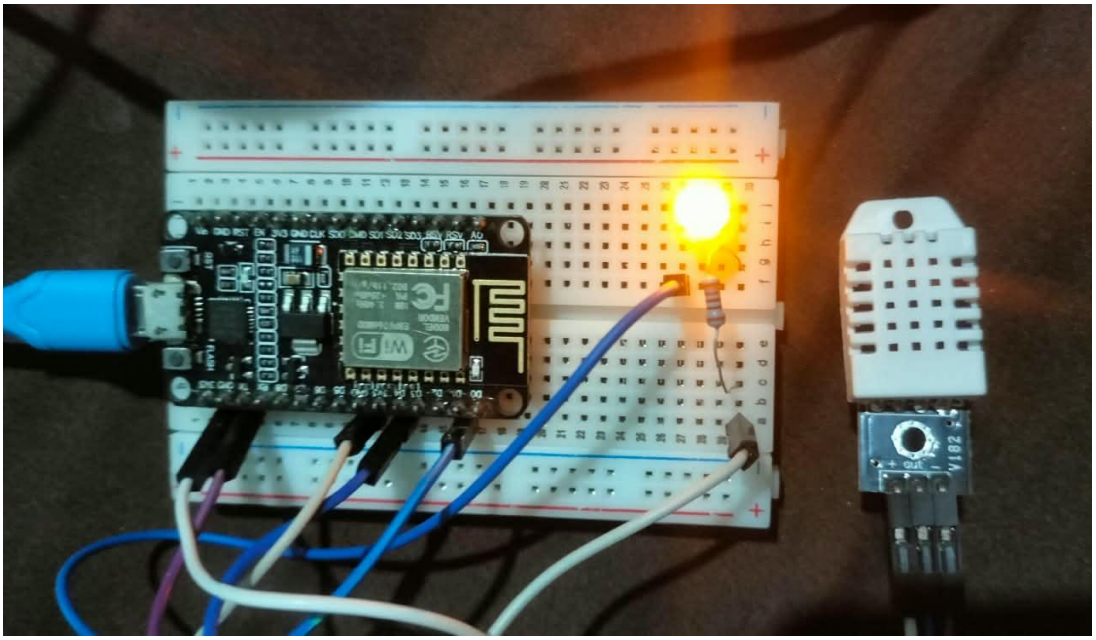
  Serial Monitor


Output
Serial Monitor
Message (Enter to send message to 'NodeMCU 1.0 (ESP-12E Module)' on 'COM5')
21:45:12.200 -> --
21:45:12.200 -> [INSTRUKSI MASUK] Topik: kampus/labiot/kendali
21:45:12.200 -> [PAYLOAD]   : {"led":1}
21:45:12.200 -> --> [AKTUATOR] Status LED diubah: MENYALA (ON)
21:45:12.200 -> --
21:45:16.147 -> [TELEMETRI TERKIRIM] {"node_id":"ESP8266_Local_Node01","suhu":31.50,"kelembapan":100.00,"status_led":1}
21:45:16.236 ->
21:45:16.236 -> --
21:45:16.236 -> [INSTRUKSI MASUK] Topik: kampus/labiot/kendali
21:45:16.236 -> [PAYLOAD]   : {"led":1}
21:45:16.236 -> --> [AKTUATOR] Status LED diubah: MENYALA (ON)
21:45:16.236 -> --
21:45:20.168 -> [TELEMETRI TERKIRIM] {"node_id":"ESP8266_Local_Node01","suhu":31.10,"kelembapan":100.00,"status_led":1}
21:45:20.326 ->
21:45:20.326 -> --
21:45:20.326 -> [INSTRUKSI MASUK] Topik: kampus/labiot/kendali
21:45:20.326 -> [PAYLOAD]   : {"led":1}
21:45:20.363 -> --> [AKTUATOR] Status LED diubah: MENYALA (ON)
21:45:20.363 -> --
21:45:24.147 -> [TELEMETRI TERKIRIM] {"node_id":"ESP8266_Local_Node01","suhu":30.80,"kelembapan":100.00,"status_led":1}
21:45:24.179 ->
21:45:24.179 ->
21:45:24.179 ->

Web NodeRed

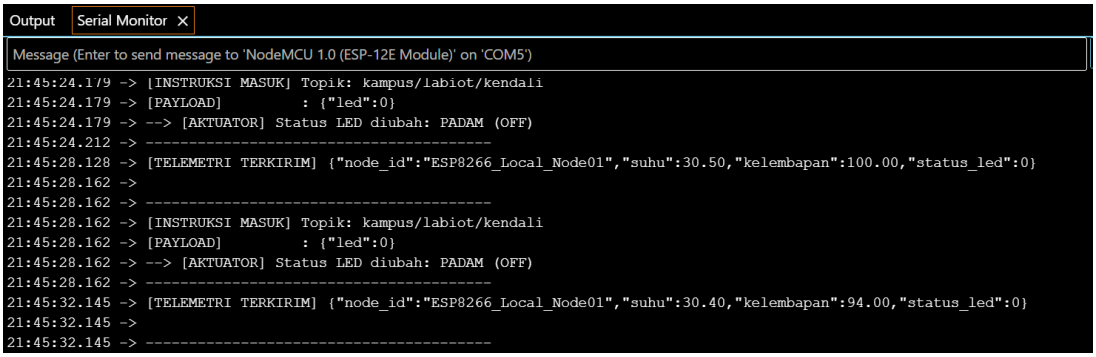


Rangkaian

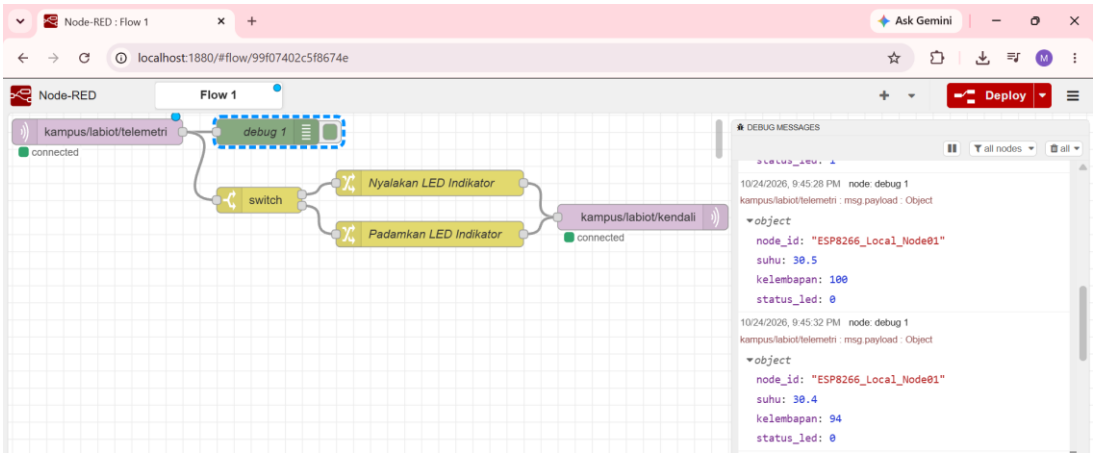


- Jika suhu berada dalam batas normal (≤〖31.0〗^∘ "C"): Node-RED memublikasikan paket JSON {"led": 0} untuk memadamkan LED indikator.

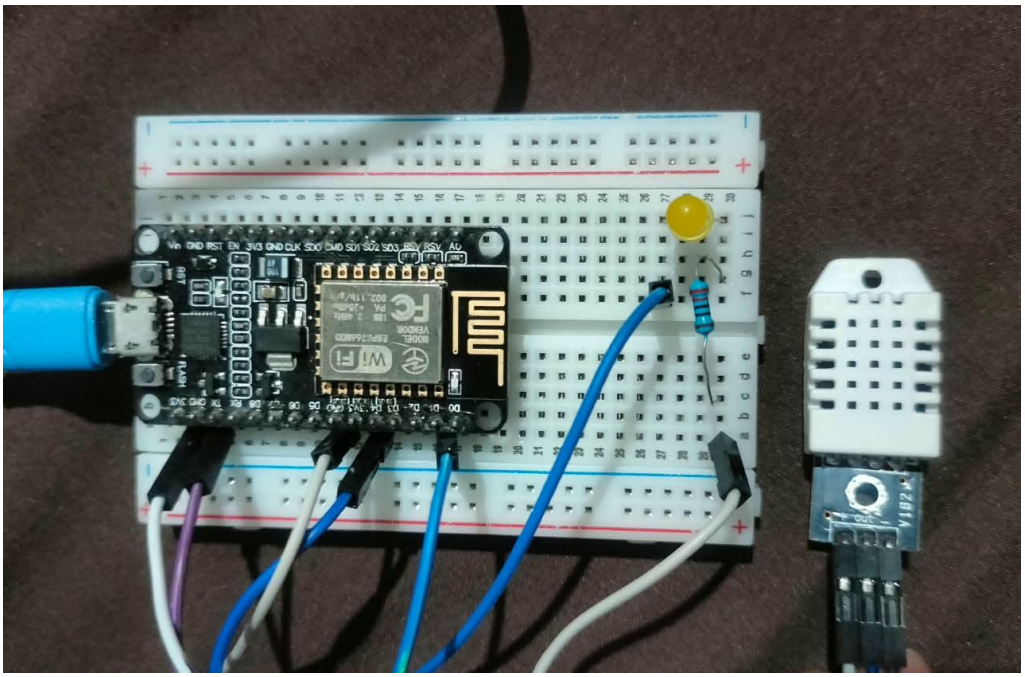
Serial Monitor



Web Node-RED



Rangkaian



7.6 Latihan

Kerjakan latihan berikut secara mandiri berbasis analisis teori dan eksperimen teknis untuk melengkapi laporan praktikum Anda:

- **Latihan 1 (Analisis Jaringan Internal Docker & Service Discovery):**
  Pada Praktikum 3, di dalam konfigurasi node mqtt in pada Node-RED, kita mengisi alamat server broker dengan nama teks **mqtt**, bukan alamat IP lokal laptop (192.168.1.15) maupun localhost.

  i. Mengapa penulisan localhost pada Node-RED di dalam kontainer Docker akan menyebabkan kegagalan koneksi (*Connection Refused*)?

  Jawab:

  Karena localhost di dalam container Node-RED mengarah ke container Node-RED itu sendiri, bukan ke laptop dan bukan ke container Mosquitto. Jadi saat Node-RED mencoba mengakses broker melalui localhost:1883, Node-RED akan mencari layanan MQTT di dalam containernya sendiri. Karena Mosquitto berada di container yang berbeda, koneksi akan gagal atau muncul *Connection Refused*.

  ii. Jelaskan bagaimana mekanisme *Docker Embedded DNS Server* dan *Bridge Network* bekerja sehingga kontainer node-red dapat menemukan kontainer mqtt-broker cukup dengan memanggil nama layanannya!

  Jawab:

  Docker Compose menempatkan container node-red dan mqtt-broker dalam jaringan internal Docker yang sama. Di dalam jaringan tersebut, Docker menyediakan Embedded DNS Server yang dapat menerjemahkan nama layanan menjadi alamat IP container secara otomatis.

  Karena nama layanan Mosquitto pada docker-compose.yml adalah mqtt, Node-RED cukup mengakses server dengan nama mqtt. Docker kemudian mencari alamat IP container Mosquitto yang sesuai melalui DNS internal dan meneruskan komunikasi melalui Bridge Network.

- **Latihan 2 (Penanganan Masalah Konektivitas Jaringan & Subnet Firewall):**
  Seorang mahasiswa mencoba menghubungkan NodeMCU ESP8266 ke broker lokal di laptopnya. ESP8266 berhasil terhubung ke Wi-Fi dan mendapatkan alamat IP 192.168.43.102, sedangkan laptop memiliki alamat IP 192.168.1.15.
  i. Mengapa NodeMCU tidak akan pernah berhasil memublikasikan data ke broker Mosquitto pada kondisi tersebut?
  Jawab:
  NodeMCU memiliki alamat IP 192.168.43.102, sedangkan laptop memiliki IP 192.168.1.15. Kedua alamat tersebut berada pada subnet yang berbeda, yaitu jaringan 192.168.43.x dan 192.168.1.x. Akibatnya, NodeMCU dan laptop tidak berada pada jaringan lokal yang sama sehingga NodeMCU tidak dapat mengakses broker Mosquitto secara langsung melalui IP laptop.
  ii. Langkah teknis apa yang harus dilakukan mahasiswa tersebut agar kedua perangkat berada dalam domain *broadcast / subnet* yang sama?
  Jawab:
  NodeMCU dan laptop harus dihubungkan ke Wi-Fi atau hotspot yang sama agar mendapatkan alamat IP pada subnet yang sama. Setelah itu, IP laptop digunakan sebagai alamat broker MQTT pada NodeMCU.

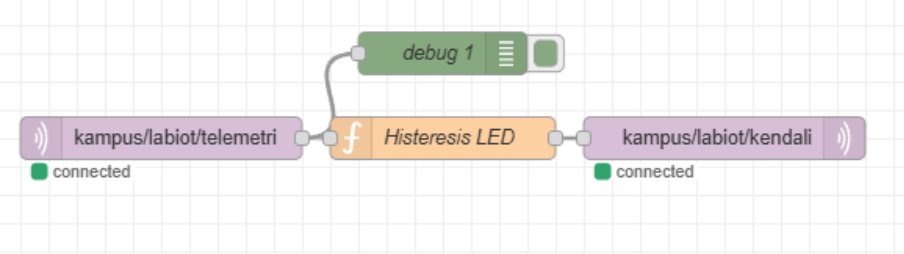
- **Latihan 3 (Implementasi Filter Histeresis pada Rule Engine):**
  Pada sistem kendali dua posisi (*Bang-Bang*), jika suhu berfluktuasi cepat di sekitar ambang batas $31.0°C$(misal: $31.05°C \rightarrow 30.95°C \rightarrow 31.02°C$), lampu LED indikator akan berkedip-kedip cepat (*flickering / hunting*) yang mengganggu stabilitas visual sistem.
  Rancang modifikasi alur Node-RED menggunakan **function node** berbahasa JavaScript yang menerapkan logika **Histeresis**:
  - LED baru menyala jika suhu $> 31.5°C$(msg.payload = {"led": 1}).

- LED baru padam jika suhu mendingin hingga < 30.0°C(msg.payload = {"led": 0}).

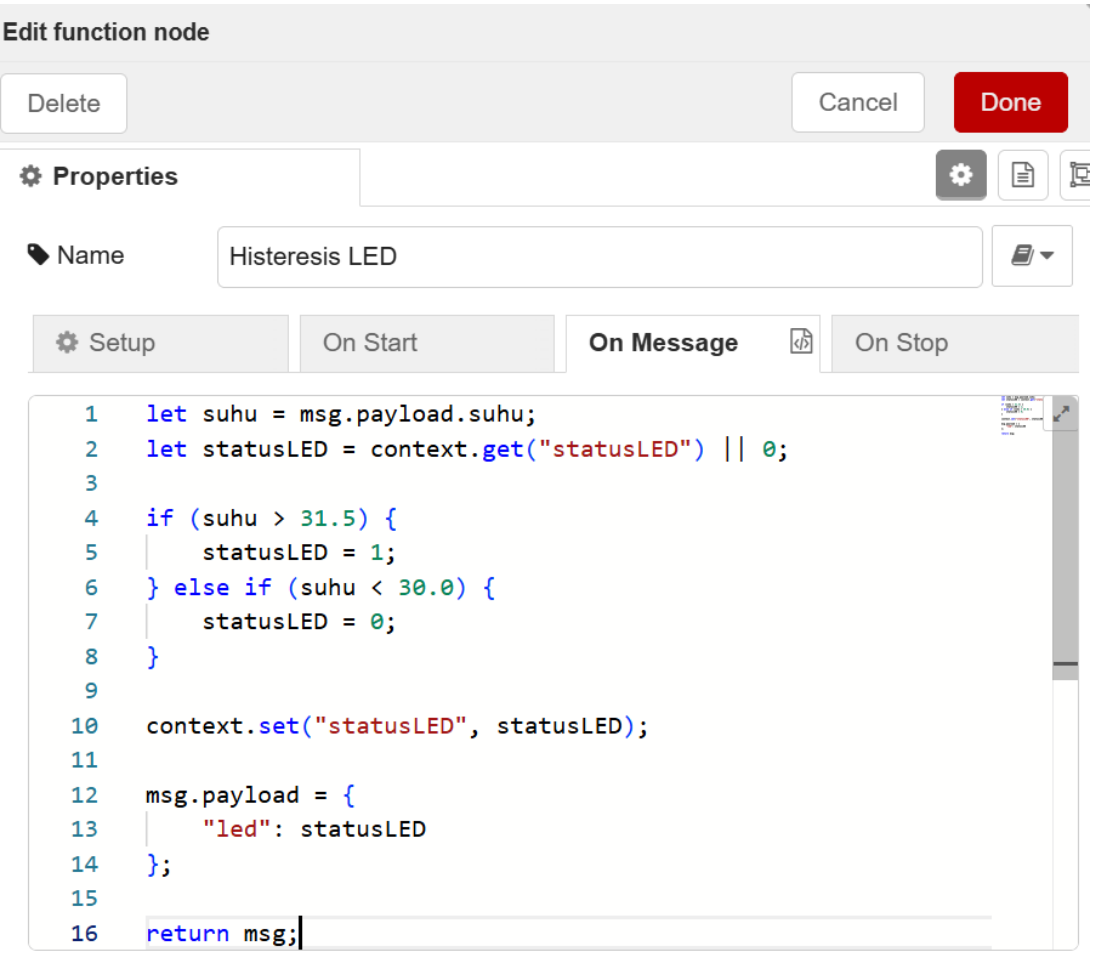
  Tuliskan kode JavaScript yang Anda masukkan ke dalam *Function Node* tersebut!

Jawab:

Alur Node-RED

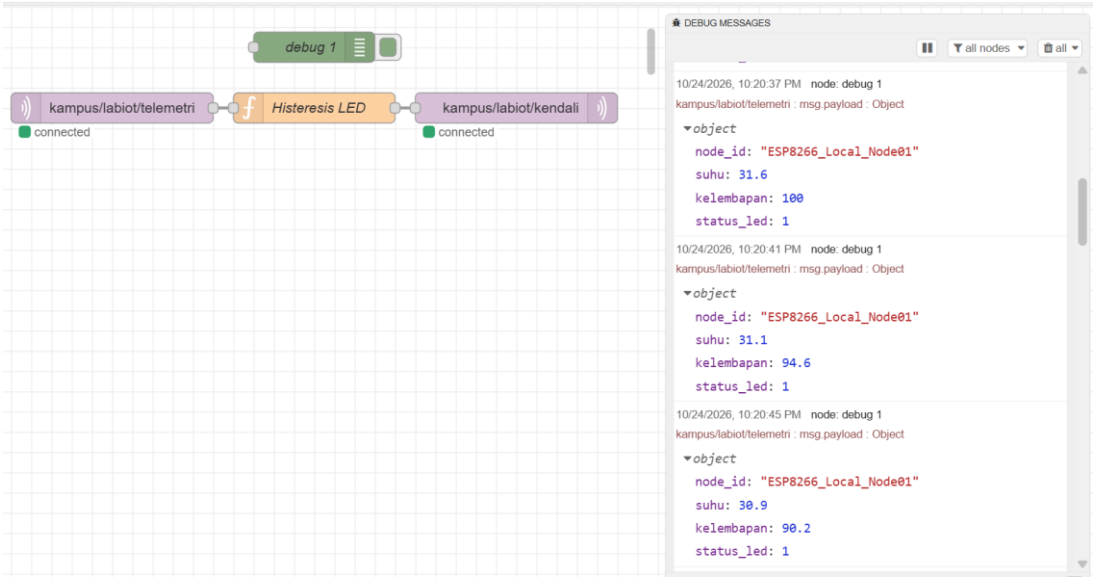


Kode Java Script

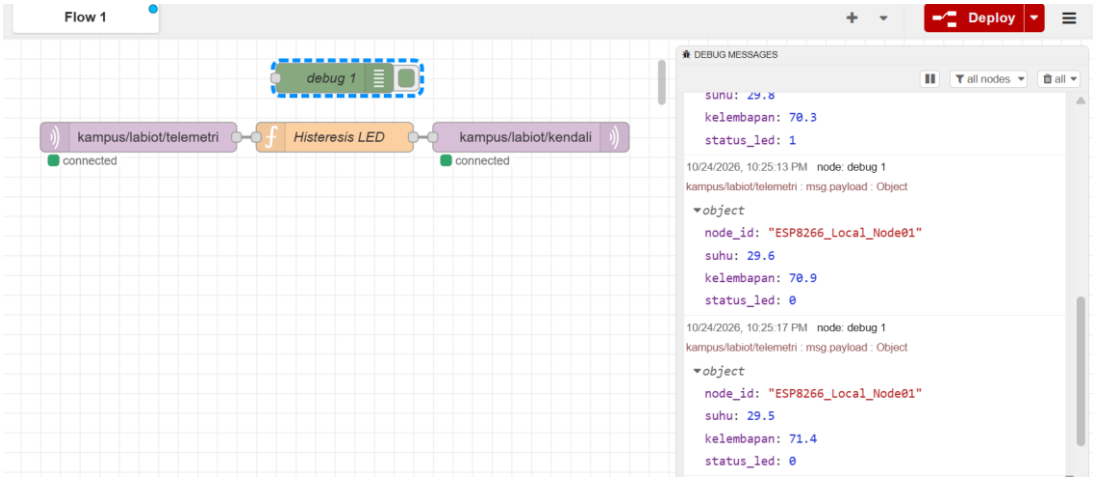


Hasil Praktikum:

- LED baru menyala jika suhu > 31.5°C(msg.payload = {"led": 1}).



- LED baru padam jika suhu mendingin hingga < 30.0°C(msg.payload = {"led": 0}).



- **Latihan 4 (Analisis Mekanisme Volume Mount & Persistensi Data):**
  Pada berkas docker-compose.yml, terdapat pemetaan direktori - ./nodered:/data dan - ./mosquitto/data:/mosquitto/data.
  i. Apa yang akan terjadi pada alur kanvas visual Node-RED yang telah Anda buat jika kontainer dimatikan dengan perintah docker compose down, kemudian dinyalakan kembali, seandainya baris volume tersebut dihapus dari docker-compose.yml?
  Jawab:
  Jika baris volume dihapus dari docker-compose.yml, data Node-RED seperti flow dan konfigurasi hanya tersimpan di dalam container. Saat

container dihapus dengan docker compose down lalu dibuat kembali, data tersebut dapat hilang karena tidak disimpan secara persisten di host.

ii. Jelaskan perbedaan mendasar antara *Bind Mount* dan *Named Volume* pada Docker!

Bind Mount menghubungkan folder tertentu yang ada pada komputer host langsung ke dalam container. Contohnya:
./nodered:/data
Sedangkan Named Volume merupakan media penyimpanan yang dibuat dan dikelola langsung oleh Docker. Pengguna cukup memberikan nama volume tanpa harus menentukan lokasi folder secara langsung pada komputer host.

7.7 Tugas: Pengamanan Server IoT Industri (MQTT Authentication & Access Control)

Instruksi Pengerjaan

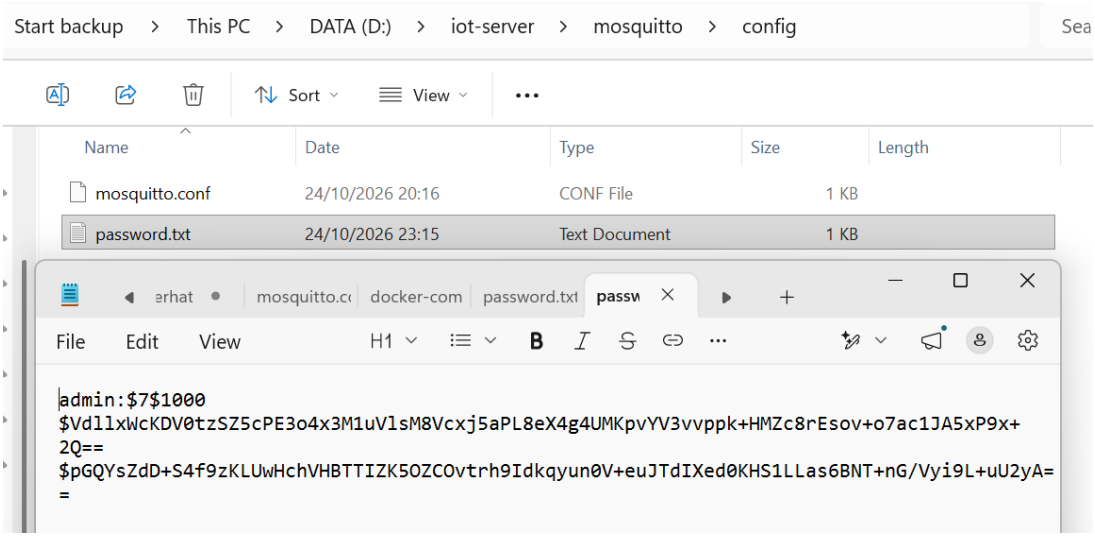
1. Membuat Berkas Kredensial Terenkripsi di Dalam Kontainer

```
D:\iot-server>docker exec -it mqtt-broker mosquitto_passwd -c /mosquitto/config/password.txt admin
Password:
Reenter password:
Adding password for user admin

What's next:
    Try Docker Debug for seamless, persistent debugging tools in any container or image → docker de
bug mqtt-broker
    Learn more at https://docs.docker.com/go/debug-cli/

D:\iot-server>
```

Buka folder iot-server/mosquitto/config/ pada laptop Anda, pastikan kini terdapat berkas baru bernama password.txt yang berisi baris *hash* kredensial akun admin.



2. Memperbarui Konfigurasi Kebijakan Mosquitto:

- Buka berkas iot-server/mosquitto/config/mosquitto.conf
- Ubah nilai allow_anonymous true menjadi: allow_anonymous false
- Tambahkan baris pemanggilan berkas kata sandi: password_file/mosquitto/config/password.txt

```
# ========================================================================
# KONFIGURASI DASAR ECLIPSE MOSQUITTO BROKER
# ========================================================================

# Listener Saluran MQTT Standar (Untuk NodeMCU ESP8266 & Klien TCP)
listener 1883
protocol mqtt

# Listener Saluran WebSockets (Untuk Aplikasi Web Browser & Dashboard Visual)
listener 9001
protocol websockets

# Perizinan Akses Awal (Tanpa Password)
allow_anonymous false
password_file /mosquitto/config/password.txt

# Konfigurasi Persistensi Penyimpanan Pesan
persistence true
persistence_location /mosquitto/data/

# Konfigurasi Berkas Log
log_dest file /mosquitto/log/mosquitto.log
log_dest stdout
```

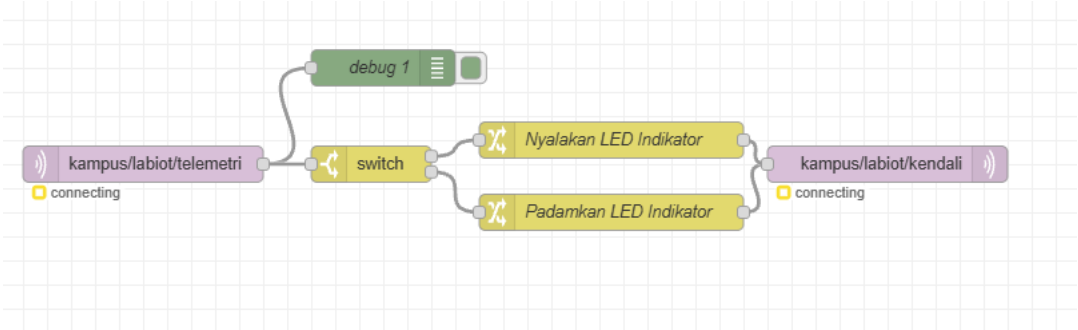
3. Memuat Ulang Layanan Broker:

```
D:\iot-server>docker restart mqtt-broker
mqtt-broker

D:\iot-server>
```

4. Analisis Dampak Kegagalan Sistem (Observasi Masalah):

Buka antarmuka Node-RED di browser, hasil yang muncul adalah connected dan warna nya kuning, menandakan belum sesuai dengan yang diminta soal:



Buka Serial Monitor NodeMCU, yang muncul adalah rc-2

```
Output  Serial Monitor ×
Message (Enter to send message to 'NodeMCU 1.0 (ESP-12E Module)' on 'COM5')
22:52:20.357 -> [MQTT] Menghubungkan ke Broker Lokal (172.20.83.244)... Gagal (rc=-2). Coba lagi dalam 4 detik...
22:52:24.393 -> [MQTT] Menghubungkan ke Broker Lokal (172.20.83.244)... Gagal (rc=-2). Coba lagi dalam 4 detik...
22:52:28.434 -> [MQTT] Menghubungkan ke Broker Lokal (172.20.83.244)... Gagal (rc=-2). Coba lagi dalam 4 detik...
```

Lalu setelah diperbaiki dengan memperbaiki permission, dengan langkah berikut

```
D:\iot-server>docker compose run --rm --user root --entrypoint sh mqtt -c "chown -R 1883:1883 /mosquitto/config /mosquitto/data /mosq
uitto/log && chmod 700 /mosquitto/config && chmod 600 /mosquitto/config/password.txt && chmod -R 755 /mosquitto/data /mosquitto/log"
Container iot-server-mqtt-run-973aef5e264c Creating
Container iot-server-mqtt-run-973aef5e264c Created

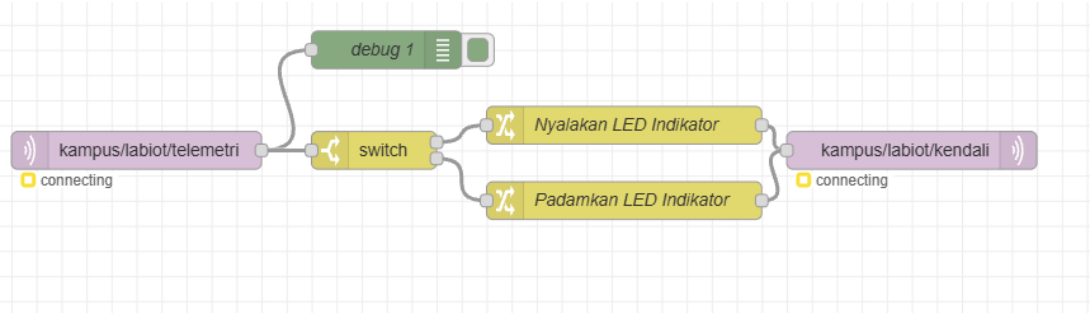
D:\iot-server>docker compose up -d
[+] up 2/2
 ✔Container node-red    Running                          0.0s
 ✔Container mqtt-broker Running                          0.0s

D:\iot-server>docker ps
CONTAINER ID   IMAGE                      COMMAND                  CREATED          STATUS                   PORTS
                                                                  NAMES
49aebc1ffa7b   eclipse-mosquitto:latest   "/docker-entrypoint.…"   10 minutes ago   Up 2 minutes             0.0.0.0:1883->1883/tcp, [:
:]:1883->1883/tcp, 0.0.0.0:9001->9001/tcp, [::]:9001->9001/tcp   mqtt-broker
ac2594530228   nodered/node-red:latest    "./entrypoint.sh"        3 hours ago      Up 3 hours (healthy)     0.0.0.0:1880->1880/tcp, [:
:]:1880->1880/tcp                                                node-red
065f74664ef5   wordpress:latest           "docker-entrypoint.s…"   11 days ago      Up 3 hours               0.0.0.0:8050->80/tcp, [::]
:8050->80/tcp                                                    wordpress
3b95cb592631   mysql:latest               "docker-entrypoint.s…"   11 days ago      Up 3 hours               3306/tcp, 33060/tcp
                                                                  mysql-wp

D:\iot-server>docker restart mqtt-broker
mqtt-broker

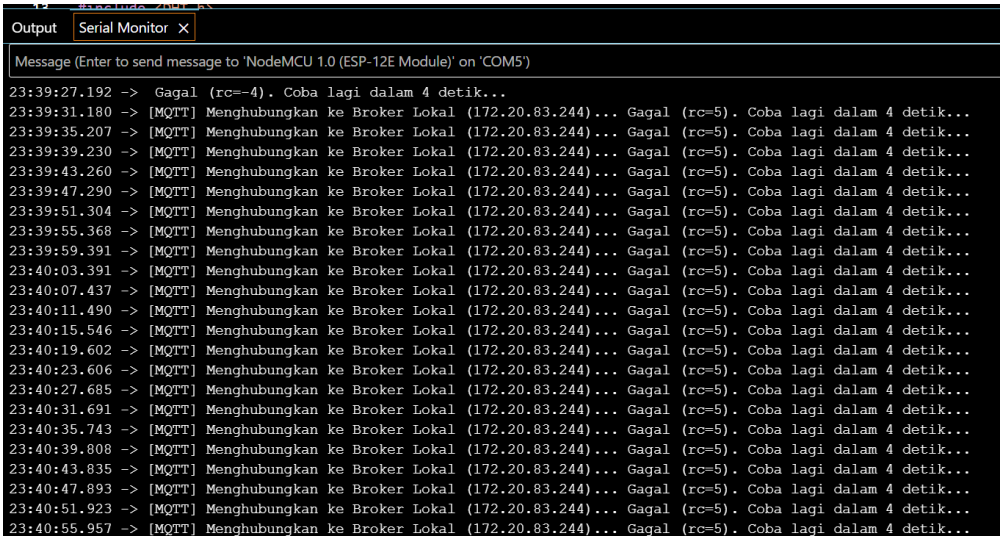
D:\iot-server>
```

- Buka antarmuka Node-RED di browser: Amati bahwa di bawah node mqtt in dan mqtt out, indikator status kini berubah menjadi warna merah bertuliskan disconnected atau connection refused.



Setelah autentikasi MQTT diaktifkan, node MQTT pada Node-RED tidak dapat terhubung dan menampilkan status connecting, yang menunjukkan Node-RED terus mencoba melakukan koneksi ulang ke broker.

- Buka Serial Monitor NodeMCU: Amati bahwa koneksi mikrokontroler kini gagal dan berulang kali menampilkan kode penolakan:



rc=5 artinya:

ESP8266 sudah berhasil mencapai broker Mosquitto, tetapi koneksinya ditolak karena belum mengirim username dan password.

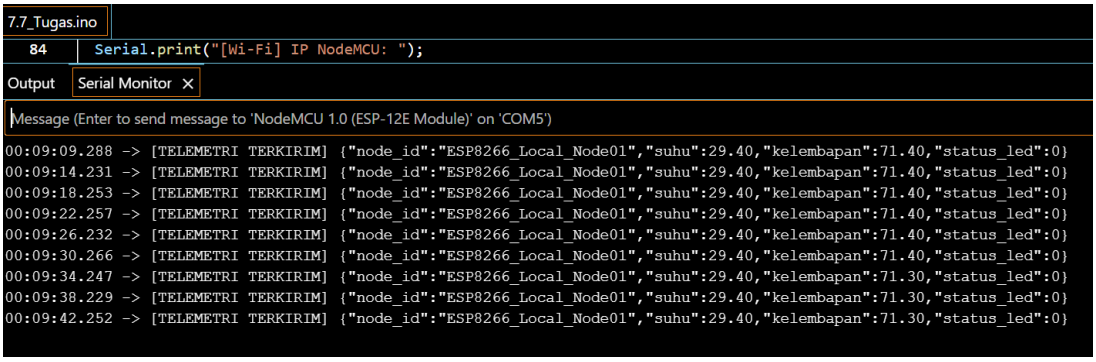
5. Rekayasa Perbaikan Sistem (Penyelesaian Tugas):

1. Pemutakhiran Skrip Program NodeMCU ESP8266:

7.7_Tugas.ino

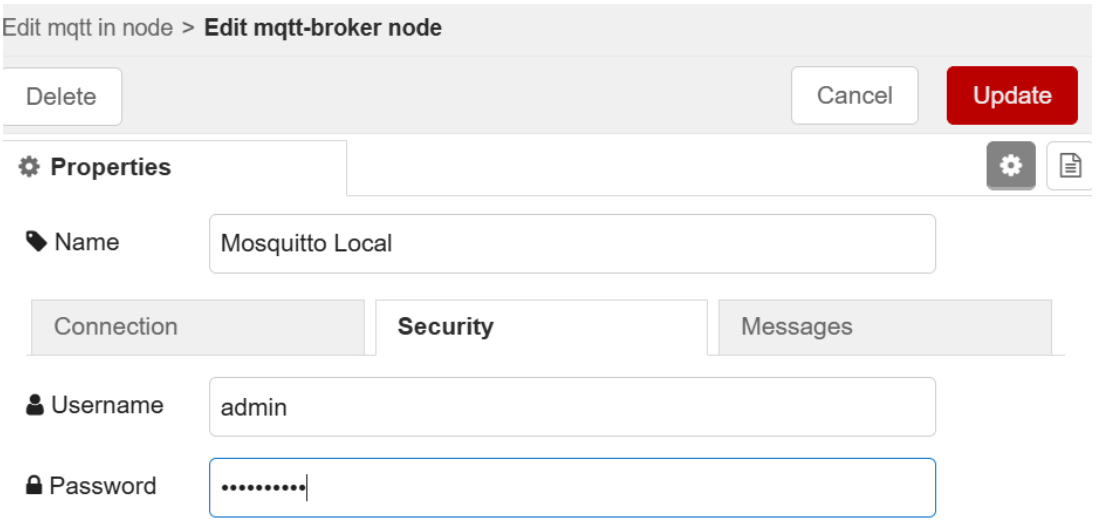
```
84    Serial.print("[Wi-Fi] IP NodeMCU: ");
85    Serial.println(WiFi.localIP());
86  }
87
88  void reconnectMQTT() {
89    while (!client.connected()) {
90      Serial.print("[MQTT] Menghubungkan ke Broker Lokal (");
91      Serial.print(mqttServer);
92      Serial.print(")...");
93
94
95      // Format: client.connect(clientId, username, password)
96      if (client.connect(clientId, "admin", "rahasia123")) {
97        Serial.println(" Terhubung dengan Autentikasi Sah!");
98        client.subscribe(topicSubCmd);
99      } else {
100       Serial.printf(" Gagal (rc=%d). Coba lagi dalam 4 detik...\n", client.state());
101       delay(4000);
102     }
103   }
104 }
105
```

Serial Monitor

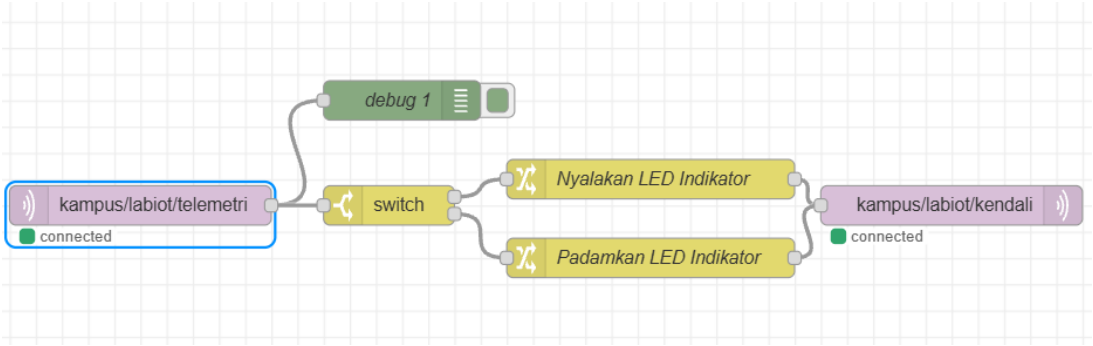


2. **Pemutakhiran Konfigurasi Kredensial pada Node-RED:**

   Klik ganda node mqtt in, lalu klik ikon pensil, lalu buka tab security dan masukkan username: admin dan password: rahasia123, seperti gambar berikut:

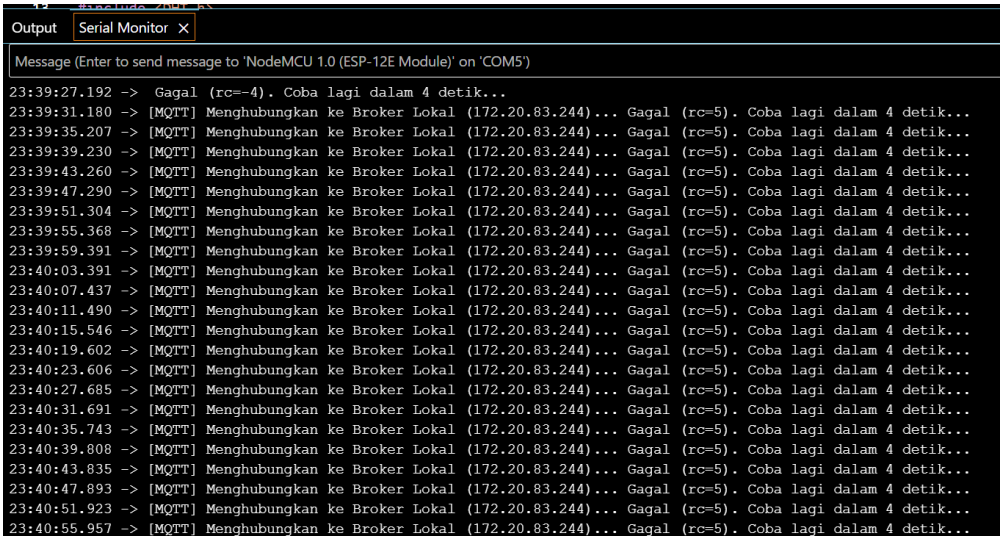


Setelah itu, klik tombol update, lalu klik done, dan tekan tombol merah deploy, maka ndikator node MQTT pada kanvas Node-RED seketika kembali berwarna hijau (connected), dan aliran data telemetri serta kendali otomasi indikator LED loop tertutup kembali berfungsi normal dan aman dari gangguan pihak luar

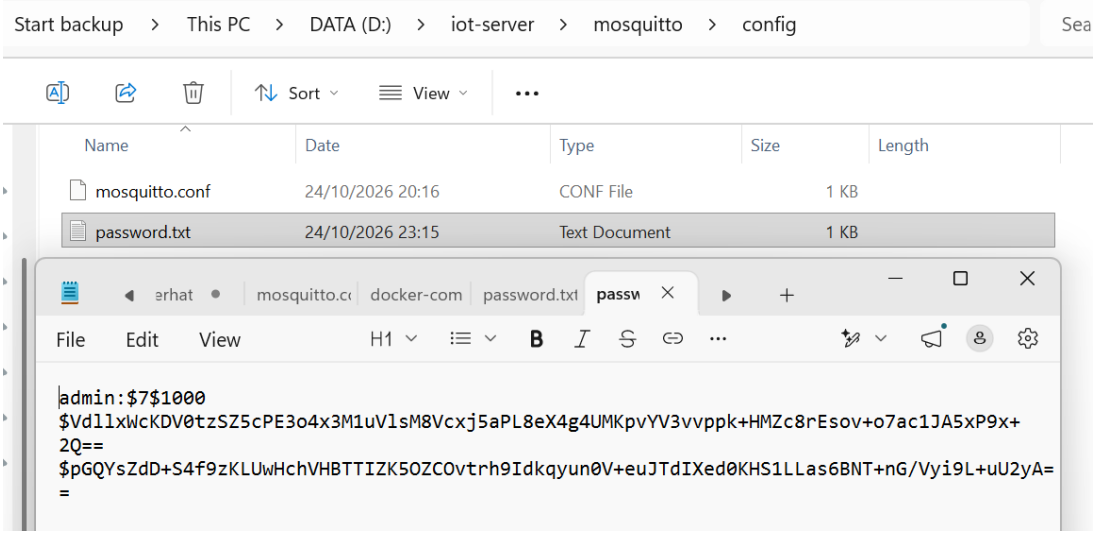


Kriteria Keberhasilan:

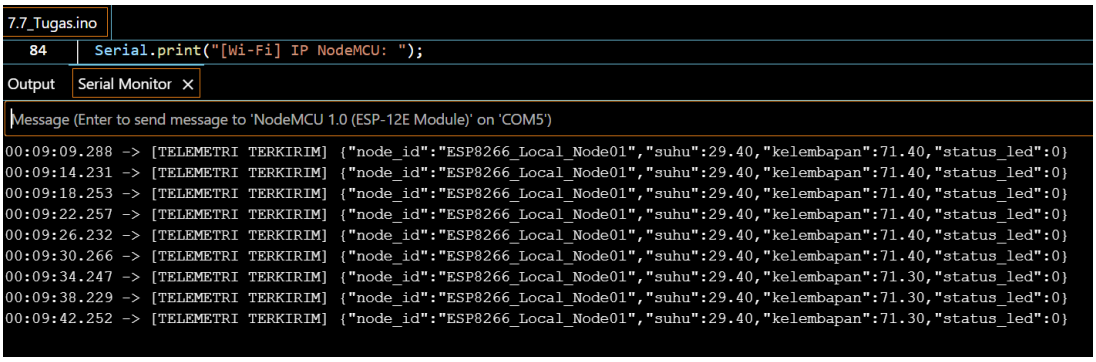
- Broker Mosquitto terbukti menolak akses anonim tanpa kredensial (allow_anonymous false).



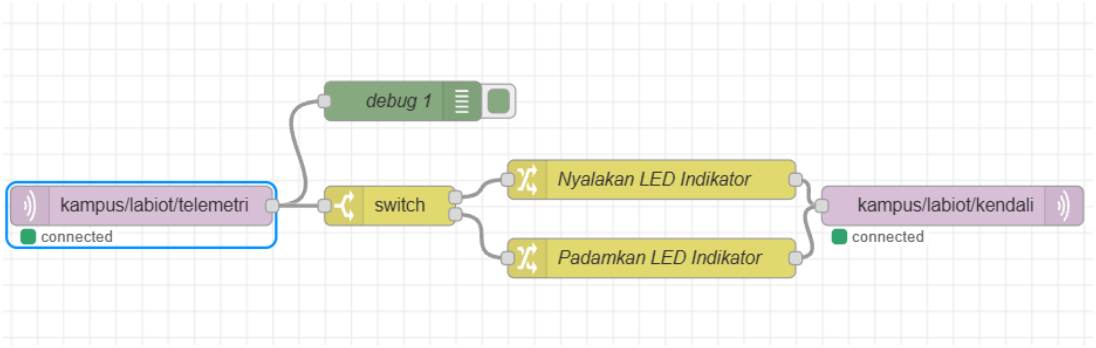
- Terdapat berkas password.txt yang memuat enkripsi kata sandi akun pengguna.



- Serial Monitor NodeMCU membuktikan pemulihan koneksi sukses menggunakan kredensial autentikasi resmi.

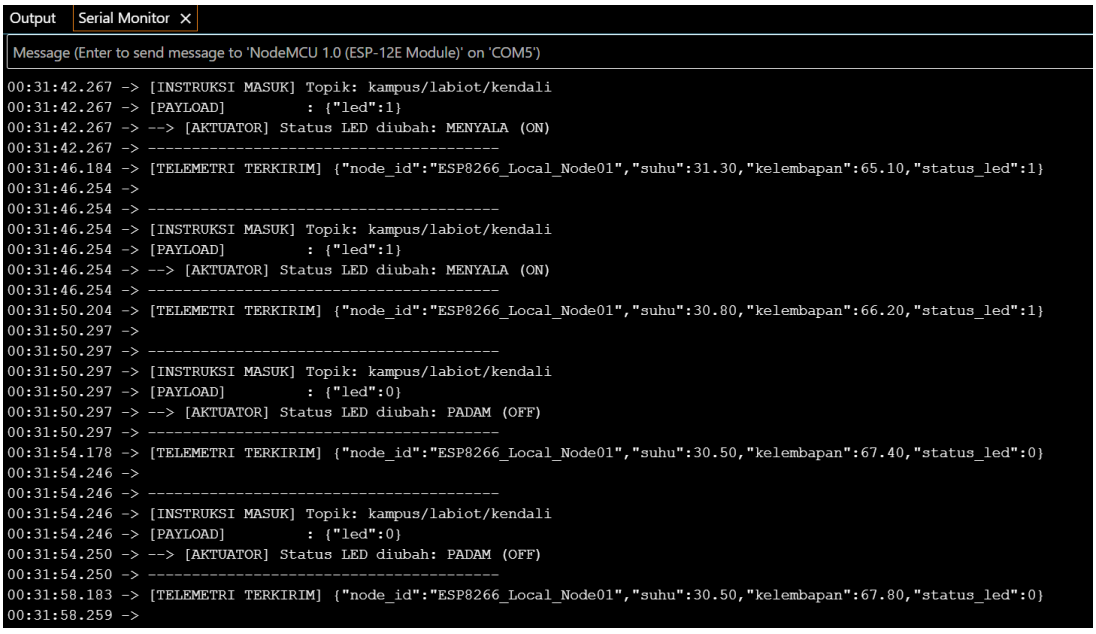


- Seluruh aliran kerja (*flow*) pada Node-RED kembali terhubung dan mampu memicu otomasi LED secara stabil.

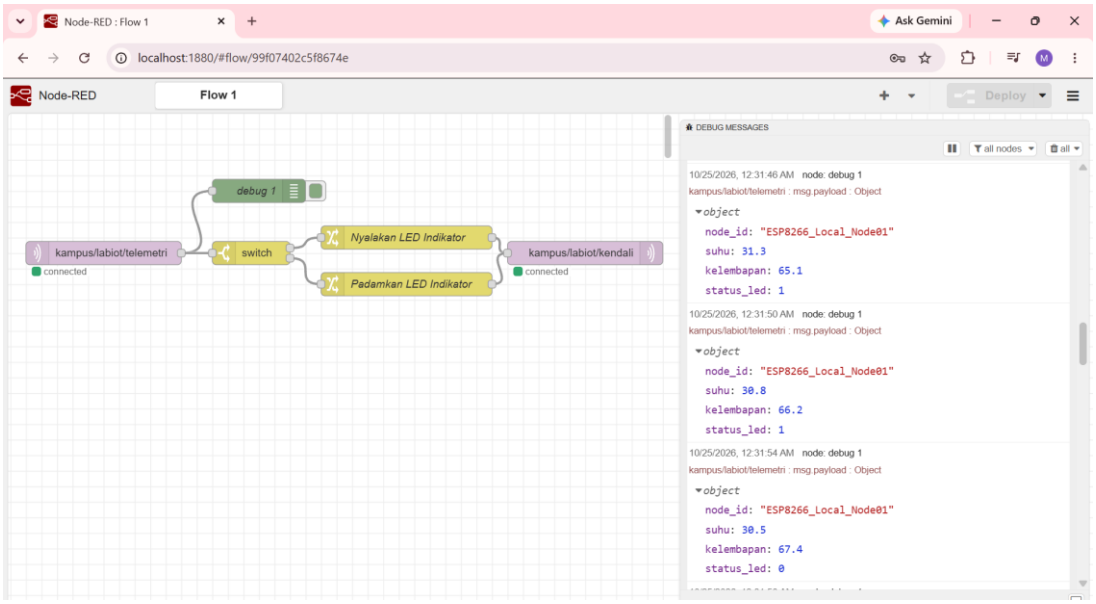


Hasil uji coba:

Serial Monitor



Web Node-RED



Berdasarkan hasil pengujian pada Node-RED, node mqtt in dan mqtt out telah berstatus connected, yang menandakan komunikasi dengan broker Mosquitto berhasil. Pada panel Debug terlihat data telemetri dari ESP8266 diterima secara real-time, seperti nilai suhu, kelembapan, dan status_led. Saat suhu mencapai 31.3°C, LED berada pada kondisi menyala (status_led: 1), sedangkan ketika suhu menurun hingga 30.5°C, LED kembali padam (status_led: 0). Hasil ini menunjukkan bahwa alur otomasi Node-RED dapat menerima data sensor dan mengendalikan LED sesuai kondisi suhu yang telah ditentukan.